

Documentazione UtenteService

Scopo della classe

`UtenteService` gestisce la logica di business relativa alla gestione dell'anagrafica utente, alle credenziali di accesso, all'aggiornamento dei ruoli e al recupero dello storico delle prenotazioni. Estende `AbstractService<Utente, UtenteDto>`, ereditando le operazioni CRUD generiche e integrando metodi personalizzati per la gestione del profilo utente.

Dipendenze

- **UtenteMapper**: converte le entità `Utente` nei rispettivi `UtenteDto` e viceversa.
- **PrenotazioneMapper**: mappa le liste di entità `Prenotazione` in liste di `PrenotazioneDto`.
- **UtenteRepository**: comunica con la base dati per persistere e recuperare le informazioni sugli utenti (tramite ID, email o nome).
- **PrenotazioneRepository**: consente di estrarre le prenotazioni create da uno specifico utente.

Costruttore

Riceve tramite Dependency Injection il repository generico, il converter, i mapper specifici (`UtenteMapper`, `PrenotazioneMapper`) e i repository dedicati (`UtenteRepository`, `PrenotazioneRepository`). Richiama `super(repository, converter)` per inizializzare la superclasse e memorizza le dipendenze nei campi della classe.

Metodi

`upgradeToAdmin(Integer id)`

Promuove uno specifico utente assegnandogli il ruolo di Amministratore.

- **Funzionalità**: ricerca l'utente per ID tramite `utenteRepository.findById(id)`. Se presente, imposta il ruolo su `RuoloEnum.ADMIN`, salva le modifiche a database e restituisce l'`UtenteDto` aggiornato. Se non trovato, lancia un'eccezione (`"User not found"`).

`findByEmail(String email)`

Ricerca un utente tramite il suo indirizzo e-mail e ne recupera lo storico completo delle prenotazioni.

- **Funzionalità**: cerca l'utente tramite `utenteRepository.findByEmail(email)`. Successivamente estrae tutte le sue prenotazioni mediante `prenotazioneRepository.findByUtenteCreatoId(...)`, le converte in DTO tramite `prenotazioneMapper.toDTOList(...)`, le imposta all'interno dell'`UtenteDto` e lo restituisce.

`findByName(String nome)`

Ricerca un utente all'interno del sistema in base al nome.

- **Funzionalità**: esegue la ricerca tramite `utenteRepository.findByName(nome)`. Se presente, converte l'entità nel rispettivo DTO e la restituisce. In caso contrario, lancia un'eccezione di tipo `ExpressionException("Name not found")`.

`cambiaPassword(String email, String password)`

Consente la modifica delle credenziali di accesso (password) per uno specifico utente.

- **Funzionalità**: individua l'utente tramite e-mail, imposta la nuova password ricevuta in input e persiste l'aggiornamento sul database tramite `utenteRepository.save(utente)`.

cambioCell(Integer idUtente, String nuovoTelefono)

Aggiorna il recapito telefonico dell'utente.

- **Funzionalità:** ricerca l'utente tramite ID, assegna il nuovo numero di telefono fornito, salva l'entità aggiornata nel database e restituisce il DTO risultante.

cambioEmail(Integer idUtente, String nuovaEmail)

Aggiorna l'indirizzo e-mail associato al profilo dell'utente.

- **Funzionalità:** individua l'utente tramite ID, aggiorna il campo e-mail con il nuovo valore fornito, salva le modifiche su database e restituisce l' `UtenteDto` aggiornato.

Flusso di esecuzione

Controller → UtenteService → UtenteRepository / PrenotazioneRepository → Database

Utente (Entity) / Prenotazioni → UtenteMapper / PrenotazioneMapper → UtenteDto → Controller → JSON

Vantaggi

Gestione centralizzata e sicura del profilo e dei privilegi utente, integrazione trasparente dello storico prenotazioni nel DTO dell'utente, conversione Entity/DTO pulita, isolamento delle operazioni di aggiornamento anagrafico e credenziali.